Parallel BFS and DFS using OpenMP

Line-by-Line Explanation

Header Files

#include <iostream>

- Used for input and output operations.
- Provides cin and cout.

#include <vector>

- Used for dynamic arrays.
- Stores adjacency list.

#include <queue>

- Used for BFS traversal.
- BFS uses queue data structure.

#include <omp.h>

- Includes OpenMP library.
- Required for parallel programming.

Namespace

using namespace std;

- Avoids writing std:: repeatedly.

Graph Class

class Graph {

- Defines Graph class.

Private Variables

int V;

- Stores number of vertices.

---

vector<vector<int>> adj;

Explanation:

- Adjacency list representation of graph.

Example:

0 → 1,2

1 → 0,3,4

Constructor

Graph(int V) : V(V), adj(V) {}

Explanation:

V(V)

- Initializes number of vertices.

adj(V)

- Creates adjacency list of size V.

Add Edge Function

void addEdge(int u, int v) {

- Function to add edge between vertices.

---

adj[u].push_back(v);

- Adds vertex v to adjacency list of u.

---

adj[v].push_back(u);

- Adds vertex u to adjacency list of v.

This creates undirected graph.

Parallel BFS Function

void parallelBFS(int start) {

- Performs Breadth First Search in parallel.

Visited Array

vector<bool> visited(V, false);

- Keeps track of visited vertices.

Initially all are false.

Queue Creation

queue<int> q;

- Queue used for BFS traversal.

Mark Start Node

visited[start] = true;

- Marks starting vertex as visited.

---

q.push(start);

- Inserts starting node into queue.

Print Message

cout << "\nParallel BFS Traversal: ";

- Displays traversal heading.

BFS Loop

while (!q.empty()) {

- Runs until queue becomes empty.

Snapshot Current Level

vector<int> level;

- Stores nodes of current BFS level.

---

while (!q.empty()) {

- Removes all nodes from queue.

---

level.push_back(q.front());

- Adds front node into level vector.

---

q.pop();

- Removes node from queue.

Parallel Processing

#pragma omp parallel for schedule(dynamic)

Explanation:

- Executes loop in parallel using OpenMP.
- schedule(dynamic) distributes work dynamically.

---

for (int i = 0; i < (int)level.size(); i++) {

- Iterates through current level nodes.

---

int node = level[i];

- Gets current node.

Traverse Neighbors

for (int neighbor : adj[node]) {

- Visits all neighboring vertices.

Check Visited

if (!visited[neighbor]) {

- Checks whether neighbor is unvisited.

Critical Section

#pragma omp critical

Explanation:

- Only one thread can enter at a time.

- Prevents race condition.

---

if (!visited[neighbor]) {

- Rechecks visited status safely.

---

visited[neighbor] = true;

- Marks neighbor as visited.

---

q.push(neighbor);

- Adds neighbor into queue.

Print Nodes

for (int node : level)

- Prints nodes of current level.

---

cout << node << " ";

- Displays BFS traversal.

---

cout << endl;

- Ends line.

Parallel DFS Utility Function

void parallelDFSUtil(int node, vector<bool> &visited) {

- Recursive DFS helper function.

Visited Flag

bool alreadyVisited;

- Stores visit status.

Critical Section

#pragma omp critical

- Ensures thread-safe access.

---

alreadyVisited = visited[node];

- Checks if node already visited.

---

if (!visited[node]) {

- If node not visited.

---

visited[node] = true;

- Marks node visited.

---

cout << node << " ";

- Prints node.

Return if Already Visited

if (alreadyVisited) return;

- Stops repeated traversal.

Traverse Neighbors

for (int i = 0; i < (int)adj[node].size(); i++) {

- Iterates through neighbors.

---

int neighbor = adj[node][i];

- Gets neighbor node.

Create Parallel Task

#pragma omp task firstprivate(neighbor)

Explanation:

- Creates separate task for neighbor.
- firstprivate gives each task its own copy.

---

parallelDFSUtil(neighbor, visited);

- Recursive DFS call.

Wait for Tasks

#pragma omp taskwait

- Waits until all child tasks finish.

Parallel DFS Function

void parallelDFS(int start) {

- Starts DFS traversal.

Visited Array

vector<bool> visited(V, false);

- Tracks visited nodes.

Print Heading

cout << "\nParallel DFS Traversal: ";

- Displays DFS heading.

Parallel Region

#pragma omp parallel

- Creates parallel thread team.

Single Thread Starts DFS

#pragma omp single nowait

Explanation:

- Only one thread starts DFS.
- nowait removes waiting barrier.

---

parallelDFSUtil(start, visited);

- Calls recursive DFS utility.

---

cout << endl;

- Ends output line.

Main Function

int main() {

- Program execution starts here.

Input Vertices and Edges

int V, E;

- Stores:
    - vertices
    - edges

---

cout << "Enter number of vertices: ";

cin >> V;

- Reads number of vertices.

Create Graph

Graph g(V);

- Creates graph object.

Read Number of Edges

cout << "Enter number of edges: ";

cin >> E;

- Reads edge count.

Read Edges

for (int i = 0; i < E; i++) {

- Loops for all edges.

---

int u, v;

cin >> u >> v;

- Reads edge vertices.

---

g.addEdge(u, v);

- Adds edge into graph.

Read Starting Vertex

int start;

- Stores starting node.

---

cout << "Enter starting vertex: ";

cin >> start;

- Reads starting vertex.

Call BFS

g.parallelBFS(start);

- Executes parallel BFS.

Call DFS

g.parallelDFS(start);

- Executes parallel DFS.

Return Statement

return 0;

- Ends program successfully.

Sample Input

6

5

0 1

0 2

1 3

1 4

2 5

0

Sample Output

Parallel BFS Traversal: 0 1 2 3 4 5

Parallel DFS Traversal: 0 1 3 4 2 5

Compile Command

g++ -fopenmp -O2 -o pbfs_dfs parallel_bfs_dfs.cpp

Explanation:

Option	Meaning
-fopenmp	Enables OpenMP
-O2	Optimization
-o	Output executable name

Line-by-Line Explanation

Header Files

#include <iostream>

- Used for input and output operations.
- Provides cout.

#include <vector>

- Used for dynamic arrays.

#include <cstdlib>

- Provides random number functions like rand().

#include <ctime>

- Used for time-related functions.
- Used with srand(time(0)).

#include <omp.h>

- Includes OpenMP library.
- Required for parallel programming.

Namespace

using namespace std;

- Avoids writing std:: repeatedly.

Array Size

#define SIZE 10000

- Defines size of array as 10,000 elements.

Correctness Check Function

bool isSorted(vector<int>& arr) {

- Checks whether array is sorted.

---

for (int i = 1; i < (int)arr.size(); i++)

- Traverses array from second element.

---

if (arr[i] < arr[i - 1]) return false;

- If current element smaller than previous:

- Array is not sorted.

---

return true;

- Returns true if array sorted correctly.

Sequential Bubble Sort

void bubbleSortSeq(vector<int>& arr) {

- Performs normal sequential bubble sort.

---

int n = arr.size();

- Stores size of array.

Outer Loop

for (int i = 0; i < n - 1; i++) {

- Controls sorting passes.

Swapped Flag

bool swapped = false;

- Tracks whether swap occurred.

Inner Loop

for (int j = 0; j < n - i - 1; j++) {

- Compares adjacent elements.

Swap Elements

if (arr[j] > arr[j + 1]) {

- Checks wrong order.

---

swap(arr[j], arr[j + 1]);

- Swaps elements.

---

swapped = true;

- Marks swap happened.

Early Exit

if (!swapped) break;

- Stops if array already sorted.

Parallel Bubble Sort

void bubbleSortParallel(vector<int>& arr) {

- Performs parallel bubble sort.

Also called:

Odd-Even Transposition Sort

---

int n = arr.size();

- Gets array size.

Main Sorting Loop

for (int i = 0; i < n; i++) {

- Runs sorting rounds.

Even Phase

#pragma omp parallel for shared(arr)

Explanation:

- Parallelizes loop using OpenMP.

---

for (int j = 0; j < n - 1; j += 2) {

- Compares pairs:

(0,1), (2,3), (4,5)

---

if (arr[j] > arr[j + 1])

- Checks wrong order.

---

swap(arr[j], arr[j + 1]);

- Swaps adjacent elements.

Odd Phase

#pragma omp parallel for shared(arr)

- Parallel odd-index comparisons.

---

for (int j = 1; j < n - 1; j += 2) {

- Compares:

(1,2), (3,4), (5,6)

Merge Function

void merge(vector<int>& arr, int l, int m, int r) {

- Merges two sorted subarrays.

Left Subarray

vector<int> left(arr.begin() + l,

arr.begin() + m + 1);

- Creates left half array.

Right Subarray

vector<int> right(arr.begin() + m + 1,

arr.begin() + r + 1);

- Creates right half array.

Indices

int i = 0, j = 0, k = l;

Explanation:

- i → left array index
- j → right array index

- k → merged array index

Merge Loop

while (i < (int)left.size() &&

j < (int)right.size()) {

- Merges arrays until one finishes.

Compare Elements

if (left[i] <= right[j])

- Selects smaller element.

---

arr[k++] = left[i++];

- Copies left element.

---

arr[k++] = right[j++];

- Copies right element.

Remaining Elements

while (i < (int)left.size())

- Copies leftover left elements.

---

while (j < (int)right.size())

- Copies leftover right elements.

Sequential Merge Sort

void mergeSortSeq(vector<int>& arr, int l, int r) {

- Performs recursive merge sort.

Base Condition

if (l < r) {

- Continues if more than one element exists.

Middle Index

int m = (l + r) / 2;

- Finds middle position.

Recursive Calls

mergeSortSeq(arr, l, m);

- Sorts left half.

---

mergeSortSeq(arr, m + 1, r);

- Sorts right half.

Merge Sorted Halves

merge(arr, l, m, r);

- Combines sorted halves.

Parallel Merge Sort

void mergeSortParallel(vector<int>& arr,

int l, int r,

int depth) {

- Performs parallel merge sort.

Base Case

if (l >= r) return;

- Stops recursion for single element.

Middle Index

int m = (l + r) / 2;

- Finds midpoint.

Sequential Fallback

if (depth <= 0) {

- Stops creating more tasks.

---

mergeSortSeq(arr, l, m);

- Sequentially sorts left half.

---

mergeSortSeq(arr, m + 1, r);

- Sequentially sorts right half.

Parallel Tasks

#pragma omp task shared(arr)

- Creates parallel task.

---

mergeSortParallel(arr, l, m, depth - 1);

- Parallel left recursion.

#pragma omp task shared(arr)

- Creates second task.

---

mergeSortParallel(arr, m + 1, r, depth - 1);

- Parallel right recursion.

Wait for Tasks

#pragma omp taskwait

- Waits until both tasks finish.

Merge Halves

merge(arr, l, m, r);

- Merges sorted halves.

Random Array Generator

void generateRandom(vector<int>& arr) {

- Generates random numbers.

---

x = rand() % 100000;

- Random values between:

0 to 99999

Main Function

int main() {

- Program execution starts here.

Array Creation

vector<int> arr(SIZE), temp;

- Creates original and temporary arrays.

Random Seed

srand(time(0));

- Initializes random generator.

Generate Random Data

generateRandom(arr);

- Fills array with random numbers.

Timer Variables

double start, end;

- Stores execution time.

Sequential Bubble Sort Timing

start = omp_get_wtime();

- Starts timer.

---

bubbleSortSeq(temp);

- Executes sequential bubble sort.

---

end = omp_get_wtime();

- Stops timer.

Print Results

cout << "Sequential Bubble Sort Time : "

- Displays execution time.

Parallel Bubble Sort

bubbleSortParallel(temp);

- Executes parallel bubble sort.

Sequential Merge Sort

mergeSortSeq(temp, 0, SIZE - 1);

- Executes sequential merge sort.

Parallel Merge Sort

#pragma omp parallel

- Creates thread team.

#pragma omp single

- Single thread starts recursion.

---

mergeSortParallel(temp, 0, SIZE - 1, 4);

- Executes parallel merge sort.

Return Statement

return 0;

- Ends program successfully.

Compile Command

g++ -fopenmp -O2 -o parallel_sort parallel_sort.cpp

Important Concepts Used

Concept	Purpose
OpenMP	Parallel programming
Bubble Sort	Simple sorting algorithm
Odd-Even Sort	Parallel bubble sort
Merge Sort	Divide and conquer sorting
omp task	Parallel recursive tasks
omp parallel for	Parallel loop execution
taskwait	Synchronization between tasks

Pr 3

You provided **two different programs** together:

1. **OpenMP Program**
    - Parallel Min
    - Parallel Max
    - Parallel Sum
    - Parallel Average
2. **CUDA Program**
    - GPU Reduction for Sum, Min, Max, Average

Both are correct as per the problem statement related to:

Parallel Reduction using OpenMP and CUDA

PART 1 — OpenMP Program Explanation

Header Files

#include <iostream>

- Used for input/output.

#include <omp.h>

- Includes OpenMP library.

#include <climits>

- Provides:

INT_MAX

INT_MIN

#include <cstdlib>

- Used for random numbers.

#include <ctime>

- Used for time seed.

Namespace

using namespace std;

- Avoids std::.

Array Size

#define N 100000

- Creates large array size.

Parallel Min Function

int findMin(int arr[], int n)

- Finds minimum element using OpenMP.

---

int minVal = arr[0];

- Initializes minimum value.

#pragma omp parallel for reduction(min:minVal)

Explanation:

OpenMP:

- Splits loop among threads.
- Each thread gets private minVal.
- Final minimum combined automatically.

---

if (arr[i] < minVal)

- Checks smaller value.

---

minVal = arr[i];

- Updates minimum.

Parallel Max Function

#pragma omp parallel for reduction(max:maxVal)

- Parallel maximum reduction.

---

if (arr[i] > maxVal)

- Checks larger value.

Parallel Sum Function

long long findSum(int arr[], int n)

- Computes sum in parallel.

#pragma omp parallel for reduction(+:total)

Explanation:

Each thread:

- Has private sum.
- OpenMP combines all sums at end.

---

total += arr[i];

- Adds elements.

Average Function

return (double)findSum(arr, n) / n;

- Average:

Sum / Number of Elements

Main Function

int* arr = new int[N];

- Dynamically allocates array.

Generate Random Numbers

arr[i] = rand() % 100000 + 1;

- Generates values:

1 to 100000

Timing

omp_get_wtime()

- Measures execution time.

Min Execution

int minimum = findMin(arr, N);

- Calls parallel min.

Max Execution

int maximum = findMax(arr, N);

- Calls parallel max.

Sum Execution

long long total = findSum(arr, N);

- Calls parallel sum.

Average Execution

double avg = findAverage(arr, N);

- Calls average function.

Free Memory

delete[] arr;

- Frees dynamically allocated memory.

OpenMP Reduction Working

Array

↓

Loop divided among threads

↓

Each thread computes private result

↓

OpenMP combines results

↓

Final output generated

PART 2 — CUDA Reduction Program Explanation

Header Files

#include <stdio.h>

#include <cuda.h>

- Includes CUDA GPU functions.

Constants

#define SIZE 1024

#define THREADS 256

Explanation:

SIZE

- Array size.

THREADS

- Threads per CUDA block.

SUM Kernel

__global__ void sumKernel(int *a, int *out)

- GPU kernel for parallel sum.

Shared Memory

__shared__ int s[THREADS];

Explanation:

- Fast shared memory inside block.
- Shared among threads.

Thread Index

int t = threadIdx.x;

- Thread number inside block.

Global Index

int i = blockIdx.x * blockDim.x + t;

- Finds array position handled by thread.

Load Data

s[t] = a[i];

- Copies array value into shared memory.

Synchronization

__syncthreads();

- Waits until all threads finish.

Reduction Loop

for (int st = blockDim.x / 2; st > 0; st /= 2)

Explanation:

Performs tree-style reduction.

Example:

256 → 128 → 64 → 32 → 16 → 8 → 4 → 2 → 1

Sum Reduction

s[t] += s[t + st];

- Adds neighboring values.

Store Block Result

if (t == 0)

- Only thread 0 stores result.

---

out[blockIdx.x] = s[0];

- Stores block sum.

MIN Kernel

if (t < st && s[t + st] < s[t])

- Finds smaller value.

MAX Kernel

if (t < st && s[t + st] > s[t])

- Finds larger value.

Main Function

int a[SIZE];

- Input array.

---

int blocks = SIZE / THREADS;

- Number of CUDA blocks.

Initialize Array

a[i] = i + 1;

- Stores:

1,2,3,4...

GPU Memory Allocation

cudaMalloc(&d_input, SIZE * sizeof(int));

- Allocates GPU memory.

Copy CPU → GPU

cudaMemcpy(d_input, a, SIZE * sizeof(int),

cudaMemcpyHostToDevice);

- Transfers data to GPU.

CUDA Events

cudaEventCreate(&evStart);

- Creates timer event.

Launch SUM Kernel

sumKernel<<<blocks, THREADS>>>(d_input, d_output);

Explanation:

Launch format:

Kernel<<<Blocks, Threads>>>

Copy Result GPU → CPU

cudaMemcpy(out, d_output,

blocks * sizeof(int),

cudaMemcpyDeviceToHost);

- Transfers results back.

Final CPU Reduction

for (int i = 0; i < blocks; i++)

- Combines partial block results.

Average

float avg = (float)sm / SIZE;

- Computes average.

Free GPU Memory

cudaFree(d_input);

cudaFree(d_output);

- Releases GPU memory.

CUDA Reduction Flow

Input Array

↓

Divide into CUDA Blocks

↓

Threads load data into shared memory

↓

Parallel reduction inside block

↓

Block results stored

↓

CPU combines block outputs

Important Concepts Used

Concept	Purpose
OpenMP Reduction	Parallel CPU computation
CUDA Kernel	GPU parallel function

Concept	Purpose
Shared Memory	Fast GPU memory
Reduction	Combine values efficiently
__syncthreads()	Thread synchronization
Parallel Sum/Min/Max	High-speed computation

Pr 4

CUDA Program 1 — Vector Addition

Line-by-Line Explanation

Header Files

#include <iostream>

- **Used for input/output operations.**

#include <cuda_runtime.h>

- **Includes CUDA runtime library.**
- **Required for CUDA functions like:**
  - **cudaMalloc()**
  - **cudaMemcpy()**

Namespace

using namespace std;

- **Avoids writing std::.**

CUDA Kernel Function

__global__ void vectorAdd(int *A, int *B, int *C, int n)

Explanation:

- **__global__**

- **Indicates CUDA kernel.**
- **Runs on GPU.**
- **Called from CPU.**

Parameters:

- **A → first vector**
- **B → second vector**
- **C → output vector**
- **n → vector size**

Calculate Thread Index

int i = blockIdx.x * blockDim.x + threadIdx.x;

Explanation:

blockIdx.x

- **Block number.**

blockDim.x

- **Threads per block.**

threadIdx.x

- **Thread number inside block.**

Formula gives unique global index.

Boundary Check

if (i < n)

- **Prevents threads from accessing invalid memory.**

Vector Addition

C[i] = A[i] + B[i];

- **Each GPU thread adds one element.**

Main Function

int main()

- **Program execution starts.**

Vector Size

int n = 1024;

- **Creates vectors of size 1024.**

CPU Memory Allocation

int *h_A = new int[n];

- **Allocates host memory for vector A.**

---

int *h_B = new int[n];

- **Allocates vector B.**

---

int *h_C = new int[n];

- **Allocates result vector.**

Initialize Vectors

for (int i = 0; i < n; i++)

- **Loops through vectors.**

---

h_A[i] = i;

- **Initializes:**

0,1,2,3...

---

h_B[i] = i * 2;

- **Initializes:**

0,2,4,6...

GPU Pointers

int *d_A, *d_B, *d_C;

- **Device pointers (GPU memory).**

GPU Memory Allocation

cudaMalloc((void)&d_A, n * sizeof(int));**

Explanation:

- **Allocates GPU memory for vector A.**

---

cudaMalloc((void)&d_B, n * sizeof(int));**

- **Allocates GPU memory for vector B.**

---

cudaMalloc((void)&d_C, n * sizeof(int));**

- **Allocates GPU memory for result vector.**

Copy CPU → GPU

cudaMemcpy(d_A, h_A,

n * sizeof(int),

cudaMemcpyHostToDevice);

Explanation:

Copies vector A from CPU to GPU.

---

cudaMemcpy(d_B, h_B,

n * sizeof(int),

cudaMemcpyHostToDevice);

- **Copies vector B.**

Block Size

int blockSize = 256;

- **256 threads per block.**

Grid Size

int gridSize = (n + blockSize - 1) / blockSize;

Explanation:

Calculates number of blocks needed.

For:

n = 1024

blockSize = 256

Result:

4 blocks

Launch CUDA Kernel

vectorAdd<<<gridSize, blockSize>>>(d_A, d_B, d_C, n);

CUDA Launch Syntax

Kernel<<<Blocks, Threads>>>

Copy GPU → CPU

```
cudaMemcpy(h_C, d_C,
        n * sizeof(int),
        cudaMemcpyDeviceToHost);
```

- **Copies result vector back to CPU.**

Print Results

```
cout << "First 10 Results:" << endl;
```

- **Displays heading.**

```
cout << h_A[i] << " + "
   << h_B[i]
   << " = "
   << h_C[i];
```

- **Prints vector addition results.**

Free GPU Memory

```
cudaFree(d_A);
```

- **Frees GPU memory.**

Free CPU Memory

delete[] h_A;

- **Frees CPU memory.**

Return Statement

return 0;

- **Ends program.**

Sample Output

0 + 0 = 0

1 + 2 = 3

2 + 4 = 6

3 + 6 = 9

CUDA Vector Addition Flow

CPU creates vectors

↓

Copy data to GPU

↓

GPU threads perform addition

↓

Copy result back to CPU

↓

Display output

CUDA Program 2 — Matrix Multiplication

Line-by-Line Explanation

Define Matrix Size

#define N 4

- **Matrix size:**

4 × 4

CUDA Kernel

__global__ void matrixMul(int *A,

int *B,

int *C)

- **GPU kernel for matrix multiplication.**

Row Index

int row = threadIdx.y;

- **Gets row number.**

Column Index

int col = threadIdx.x;

- **Gets column number.**

Sum Variable

int sum = 0;

- **Stores multiplication result.**

Matrix Multiplication Loop

for (int k = 0; k < N; k++)

- **Traverses row and column.**

Multiplication Formula

sum += A[row * N + k] *

B[k * N + col];

Explanation:

Implements:

C[row][col] =

Σ A[row][k] × B[k][col]

Store Result

C[row * N + col] = sum;

- **Stores result into output matrix.**

Matrix Declaration

int A[N][N], B[N][N], C[N][N];

- **Input and output matrices.**

Initialize Matrices

A[i][j] = i + j;

Example:

0 1 2 3

1 2 3 4

B[i][j] = i * j;

Example:

0 0 0 0

0 1 2 3

GPU Memory Allocation

```
cudaMalloc((void**)&d_A,
    sizeof(int) * N * N);
```

- **Allocates GPU memory.**

Copy Matrices to GPU

```
cudaMemcpy(d_A, A,
    sizeof(int) * N * N,
    cudaMemcpyHostToDevice);
```

- **Copies matrix A to GPU.**

Thread Configuration

dim3 threadsPerBlock(N, N);

Explanation:

Creates:

4 × 4 threads

Each thread computes one matrix element.

Launch Kernel

matrixMul<<<1, threadsPerBlock>>>

(d_A, d_B, d_C);

Explanation:

- **1 CUDA block**
- **4×4 threads**

Copy Result Back

cudaMemcpy(C, d_C,

sizeof(int) * N * N,

cudaMemcpyDeviceToHost);

- **Transfers result matrix to CPU.**

Print Matrix

cout << C[i][j] << " ";

- **Displays matrix multiplication output.**

Free GPU Memory

cudaFree(d_A);

cudaFree(d_B);

cudaFree(d_C);

- **Releases GPU memory.**

Matrix Multiplication Flow

CPU initializes matrices

↓

Copy matrices to GPU

↓

Each GPU thread computes one element

↓

Store result matrix

↓

Copy result back to CPU

Important CUDA Concepts Used

Concept	Purpose
CUDA Kernel	GPU function
Threads	Parallel execution
Blocks	Group of threads
cudaMalloc	Allocate GPU memory
cudaMemcpy	Transfer data
threadIdx	Thread index
blockIdx	Block index
Matrix Multiplication	Parallel computation